QwenGyre: An Elastic Reinforcement Learning Framework for Training xLong-Horizon Agents

Weiqi Wang* Yuxin Zhou* Mouxiang Chen* Siyuan Zhang
Yi Zhang Yuyan Luo Zhiyu Yin Chencan Wu
Jiemin Jiang Wentao Yao Chujie Zheng JianWei Zhang†

Alibaba Token Hub, Alibaba Group

arXiv:2609.33848v2 [cs.LG] 20 Oct 2026

Abstract

Large language model (LLM) agents increasingly undertake eXtreme-long (xLong) horizon tasks, where a single execution can span hours, hundreds of model–environment interactions, and nearly 1M tokens per rollout. Applying online reinforcement learning (RL) to such executions poses two fundamental challenges: (1) severe execution variance and prolonged rollout delays cause massive GPU idling; and (2) complex non-linear branching generates massive trajectory redundancy, crippling training efficiency.

To address these, we presents QwenGyre, an end-to-end framework for xLong-horizon online RL. QwenGyre elastically reallocates GPUs between rollout and training without interrupting live executions, while its trajectory processor reconstructs branching histories, scores partial progress, and deduplicates redundant paths to bound training costs. Scaled to our flagship model, Qwen 3.8 2.4T, with 700K tokens per rollout, QwenGyre yields a 6.0% absolute gain on NL2RepoBench (52.5% → 58.5%) in 48 steps. Across our evaluations on diverse domains of training datasets, QwenGyre delivers up to 1.85× and 1.78× speedups over Colocate and Async, respectively.

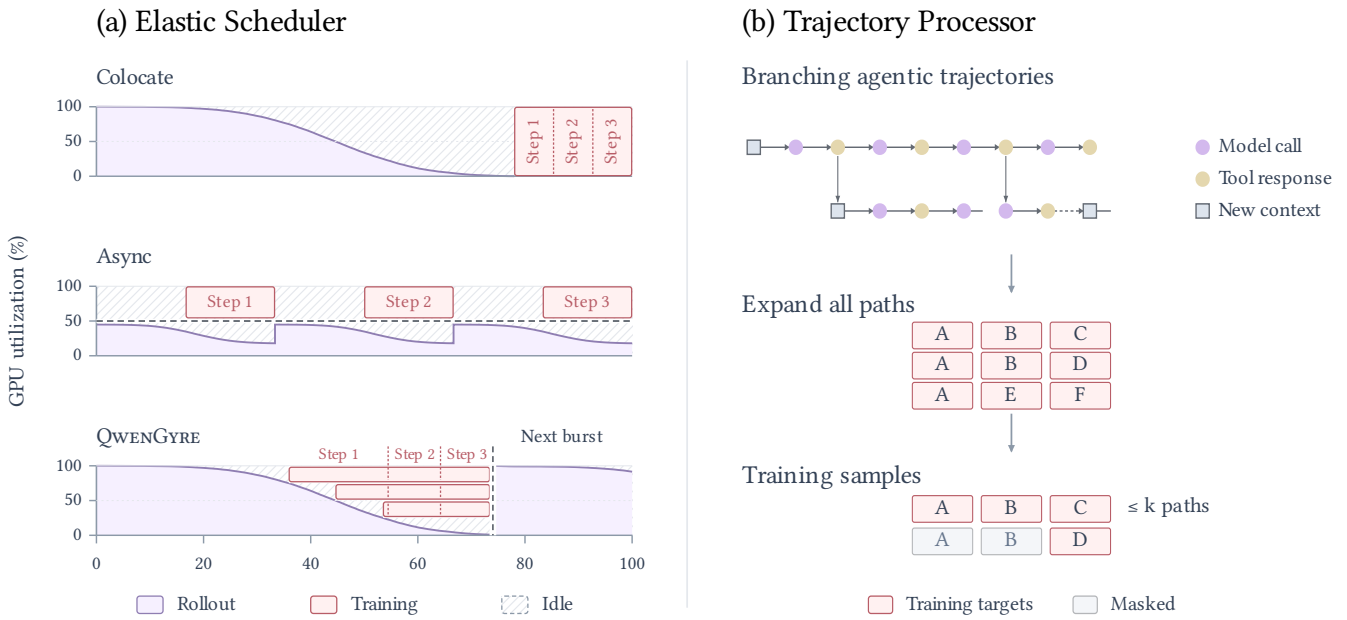


Figure 1: QwenGyre addresses two challenges of xLong-horizon agentic RL. (a) The Elastic Scheduler reallocates idle rollout capacity to training while continuing to serve ongoing executions. (b) The Trajectory Processor converts branching executions into bounded training samples while masking repeated targets in shared prefixes.

*Equal contribution.
†Corresponding author.

1 Introduction

Large language model (LLM) agents increasingly solve software engineering tasks that are *long-horizon*: completing one task requires an extended sequence of interdependent decisions and environment interactions over a persistent workspace. A representative class of such tasks is issue resolution (e.g., SWE-bench (Jimenez et al., 2024; Deng et al., 2026)), which localizes a single fix within an existing repository and spans tens of interactions. On these workloads, online reinforcement learning (RL) has demonstrated promising gains by iteratively refining policies through environment interactions (Luo et al., 2025a; Golubev et al., 2025; Song et al., 2026b; Du et al., 2026b).

As foundation models rapidly advance in reasoning and context capabilities, the paradigm is shifting toward fully autonomous, end-to-end software development, such as repository generation (Ding et al., 2026), system reimplementation (Yang et al., 2026), and codebase-wide refactoring (Hong et al., 2026). These workloads extend long-horizon execution to multi-hour, project-scale runs. A single stateful rollout spans hundreds of dependent model–environment interactions and processes around 1M tokens across its context. We refer to this regime as *xLong-horizon*. Scaling RL to xLong-horizons introduces fundamental challenges on both system scheduling and trajectory modeling.

Challenge 1: Extreme rollout variance and severe GPU underutilization at xLong-horizons. Online RL conventionally adopts a *colocate* design that alternates the GPU pool between rollout and training (Sheng et al., 2025; Mei et al., 2025). At xLong-horizons, however, rollouts exhibit severe long-tail variance; without general support for checkpointing workspace state, training remains blocked until all live executions end, leaving GPUs idle on a few trailing tasks. Alternative *async* deployments statically partition GPUs into dedicated pools (Fu et al., 2025). Nevertheless, at xLong-horizons, the wait for a ready batch can far exceed the duration of a training step, leaving training GPUs idle while rigid partitions prevent either stage from utilizing the other's spare capacity. The challenge is therefore to reallocate GPUs dynamically as demand changes, using idle capacity for training while maintaining inference service for ongoing rollouts.

Challenge 2: Complex trajectory branching and prohibitive training overhead at xLong-horizons. The linear agent loop used in simple agent designs (Yao et al., 2023; Yang et al., 2024) does not survive at xLong-horizons: sustaining hours of execution typically relies on black-box harnesses that routinely compact saturated histories, delegate sub-tasks to sub-agents with isolated contexts, and retry failed execution paths (He et al., 2026b; Song et al., 2026a). As a result, a single task execution no longer yields a sequential trajectory, but an intricate, non-linear graph of divergent context paths. This non-linearity leads to an explosion in sample and token volume: expanding all these paths into individual training samples introduces severe redundancy across shared prefixes and incurs prohibitive training overhead. The challenge is thus to reconstruct valid training representations from such complex, branching executions while preventing the sheer volume of paths and tokens from overwhelming the training pipeline.

In this paper, we present QwenGyre, an end-to-end framework for online RL through black-box agent harnesses at xLong-horizons. Its design separates the lifetimes of harness executions, GPU roles, and training samples. Harness state persists across GPU role changes, while recorded model calls are materialized into training samples on demand.

To address Challenge 1, QwenGyre introduces an *elastic scheduler*, which adjusts GPU allocation between rollout and training according to the amount of unfinished rollout work while ensuring that model calls from ongoing harness executions continue to be served. Centralized data-parallel training allows newly available nodes to join an ongoing batch as the training pool grows. Streaming training distributes work according to each participating data-parallel group's progress, helping the groups finish the same batch at similar times to minimize idle gaps.

To address Challenge 2, QwenGyre introduces a *trajectory processor*, which organizes each execution's recorded model calls into a trajectory tree with shared prefixes, preserving each output's original conditioning context. Task-specific evaluation scores preserved work after termination, including assessable partial progress after timeout. Once valid rewards are available, QwenGyre selects a bounded set of trajectories per execution by role priority to control training cost. Training uses only policy-generated targets (Gallouédec & Rasul, 2026), counts shared targets once, and averages token losses within each execution to avoid overweighting executions with more paths.

We evaluate QwenGyre on NL2RepoBench (Ding et al., 2026), DeepSWE (Huang et al., 2026), and Terminal-Bench (Merrill et al., 2026) using Qwen 3.6 122B. We also apply QwenGyre to xLong RL training of our flagship model, Qwen 3.8 2.4T, on NL2RepoBench. We compare against Colocate and Async baselines using equal GPU budgets under the same staleness constraints. Across the reported configurations, QwenGyre achieves end-to-end speedups of 1.38 to 1.78 times over Async and 1.21 to 1.85 times over Colocate while matching baseline training scores. On NL2RepoBench, QwenGyre improves Qwen 3.8 2.4T's score from approximately 52.5 % to 58.5 % over 48 training steps.

In summary, this paper makes the following contributions:

- We identify the scheduling and trajectory-processing challenges of online RL for xLong agent executions.
- We design an elastic scheduler that reallocates GPUs between rollout and training as demand changes, lets newly available GPUs join an ongoing training batch, and balances training work across them to reduce idle time.
- We develop a trajectory processor that preserves each model output's original context and evaluates partial work after timeouts. It selects a limited number of trajectories per execution, counts shared outputs once,

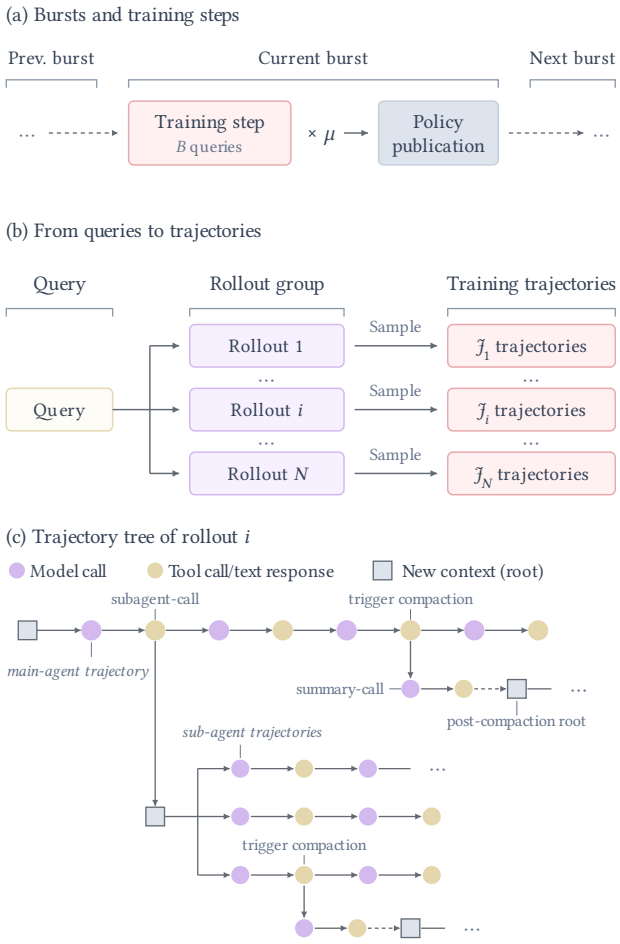


Figure 2: Hierarchy of training bursts, rollout groups, and trajectories. (a) Each burst performs μ consecutive training steps, then publishes the policy; each step consumes B query groups. (b) Each query produces N independent rollouts, and rollout i yields J_i training trajectories. (c) A rollout includes main-agent and sub-agent trajectories, with compaction producing summary branches. Dashed links reinsert summaries into post-compaction contexts.

and averages token losses within each execution.

- We demonstrate QwenGyre on Qwen 3.8 2.4T, improving NL2RepoBench passrate from 52.5% to 58.5% in 48 training steps. Across evaluated workloads, QwenGyre achieves up to 1.85× and 1.78× end-to-end speedups over Colocate and Async, respectively.

2 Background and Motivation

2.1 xLong Agentic RL Workloads

We first describe the group-relative RL pipeline and the execution characteristics of xLong agentic workloads. We then examine their implications for rollout–training scheduling and training data construction.

Group-relative RL pipeline. For each *query*, N independent rollouts sampled from the policy form a *rollout group* (Figure 2(b)). Advantages are computed from the group's scalar rewards, avoiding a learned value function (Shao et al., 2024). In our pipeline, each *training step* consumes a *batch* of B distinct rollout groups and advances the policy version by one. To amortize the cost of weight publication, the RL framework may perform μ consecutive training steps between publications; we call this interval a *burst*. Each burst consumes μB rollout groups and publishes the weights at its final step, as shown in Figure 2(a).

Harness-driven agentic RL An agentic rollout is an end-to-end task execution in which a model interacts with an environment through a harness. The harness maintains task state, invokes tools according to model outputs, and constructs context for subsequent model calls. We call each model call–response interaction a *model calls*, each tool invocation a *tool call*, and the full task lifecycle, including retries, a *rollout execution*. Figure 2(c) shows an execution that branches at tool calls and the trajectories selected from it for training.

In our black-box setting, the harness accesses the model through a proxy provided by the RL framework. The framework records model calls and receives task-level evaluation results without changing the harness's execution logic (He et al., 2026b; Song et al., 2026a). This interface provides no general mechanism to checkpoint and resume a live execution. If a model call cannot be served, the harness may stall or time out, changing the task outcome for reasons unrelated to the policy's decisions. Scheduling must therefore preserve inference access throughout the execution. Requests can be routed to different rollout instances while the harness continues running.

The xLong execution profile. Recent software engineering benchmarks evaluate agents on repository construction, program reimplementation, and migration across entire codebases (Ding et al., 2026; Yang et al., 2026; Desai et al., 2026; Hong et al., 2026). We focus on xLong executions that last hours, span hundreds of dependent model–environment interactions, and process around 1M input and output tokens across model calls. This lifecycle volume counts repeated contexts at each request and can exceed the context limit of an individual request.

These workloads exhibit both long execution tails and clustered terminations. Execution durations vary across queries and within each rollout group: generation length and tool latency affect the time per interaction (Zhang et al., 2026b). Over hundreds of dependent interactions, these differences can leave some executions running long after their peers finish. Since group-relative advantages require all N rewards, a straggler can delay training on its entire group. Executions with similar start times and a common timeout budget may expire within a narrow interval, producing clusters of execution terminations that can sharply reduce inference demand.

2.2 Rollout–Training Scheduling

RL frameworks organize rollout and training through two scheduling policies. A data *dispatch policy* controls

when and how much rollout work enters the pipeline. A GPU *placement policy* allocates GPUs between rollout and training in space and time. These policies can be combined to form different framework designs: Async, for example, can use either continuous or boundary dispatch.

Data dispatch policy and staleness. Under *boundary dispatch*, each rollout group enters the dispatch queue when admitted and retains its slot until training consumes it. Let $\varphi \geq 0$ denote the extra dispatch in units of B rollout groups. With μ training steps per burst, the target queue depth is $(\varphi + \mu)B$ groups. The queue is filled to this depth at startup and replenished with μB new groups after each burst's weight publication.

We measure a group's *scheduling staleness* as $d = v_t - v_d$, with policy versions indexed by training steps. Here, v_d is the version of the latest published policy when the group is admitted, and v_t is the policy version immediately before the step that consumes it.

The target queue depth Q_b and steady-state mean scheduling staleness satisfy

$$Q_b = (\varphi + \mu)B, \tag{1}$$

$$\mathbb{E}[d] = \varphi + \frac{\mu - 1}{2}. \tag{2}$$

The φ term accounts for extra dispatched groups, while $\frac{\mu-1}{2}$ averages the training-step positions within a burst, indexed from zero.

GPU placement policy and utilization. *Async* assigns fixed GPU pools to rollout and training, allowing the two stages to overlap (Fu et al., 2025; Zhong et al., 2025). Long rollout latency requires high concurrency to sustain throughput, but increasing dispatch depth also raises scheduling staleness (Equation 2). Limiting dispatch depth can therefore leave training GPUs waiting for data, while the fixed split prevents training from using spare rollout capacity as executions finish (Figure 1, Async).

Colocate alternates a shared GPU pool between rollout and training, as supported by HybridFlow and ReaL (Sheng et al., 2025; Mei et al., 2025). Because the pool switches as a unit, training must wait for the rollout tail even when groups are ready and GPUs are idle (Figure 1, Colocate). More steps per burst amortize switching and weight-publication costs, but increase scheduling staleness for later steps.

2.3 From Harness Executions to Training Data

At xLong-horizons, context management and sub-agent delegation routinely break the append-only history of a simple agent loop (Yao et al., 2023; Yang et al., 2024). Harnesses compact histories as context windows fill and maintain separate contexts for sub-agents.

The harnesses used for these executions are also used in deployment, and the most capable, such as Claude Code and Codex, are closed and evolve rapidly; reimplementing their control flow inside an RL framework is impractical and would train the policy under a harness different from the one it is deployed with. This motivates the black-box interface of Section 2.1, which exposes only model calls and task-level evaluation results. Using these records for group-relative RL requires valid outcome rewards and explicit choices of training targets and loss weights.

Execution structure. An execution can contain multiple *trajectories*. A trajectory is a linear path of model calls in which each successive call extends the preceding call's recorded context and response. Compaction and other context edits can rewrite an existing history (He et al., 2026b). Calls may remain causally related across these changes without forming one append-only history. A single agent or role can therefore contribute multiple trajectories. Concatenating requests across these boundaries as one history can give responses contexts they did not have during generation. Across hundreds of interactions, repeated context changes produce varying numbers of trajectories with different lengths.

Reward validity. An execution's reward can fail to support training in two distinct ways. First, an execution may reach its deadline before completing the task (Desai et al., 2026): its captured trajectories describe a partial attempt, and assigning all partial attempts the same failure reward obscures differences in task progress. Second, infrastructure or evaluator failures may leave no valid assessment at all; a missing assessment is distinct from a valid zero reward and cannot be filled in as one. Even after every execution in a group has terminated, the group may therefore lack the N valid sample rewards needed for advantage estimation.

Training semantics. A task-level outcome reward evaluates an execution of the original task. Figure 2 illustrates the varying trajectory counts: each query has N executions, while execution i contributes J_i selected trajectories. A flat average of trajectory losses gives executions with more selected paths greater aggregate weight (He et al., 2026b). Within an execution, the same averaging can give many short auxiliary paths more aggregate weight than a long main-agent path. A shared response can likewise receive extra weight if each copy contributes to the loss. The RL objective must therefore specify which roles, trajectories, and tokens contribute to the loss, together with the normalization and weighting rules.

3 QwenGyre Overview

QwenGyre supports online RL through unmodified black-box agent harnesses at xLong-horizons. Its architecture consists of two cooperating parts (Figure 3). The elastic scheduler (Section 4) reallocates GPUs between rollout and training while allowing live harness executions to continue. The trajectory processor (Section 5) constructs training samples from recorded model calls, retaining execution records independently of the samples admitted for optimization.

Elastic scheduler. GPU resources comprise elastic cells and an optional standalone rollout pool. Cells share a

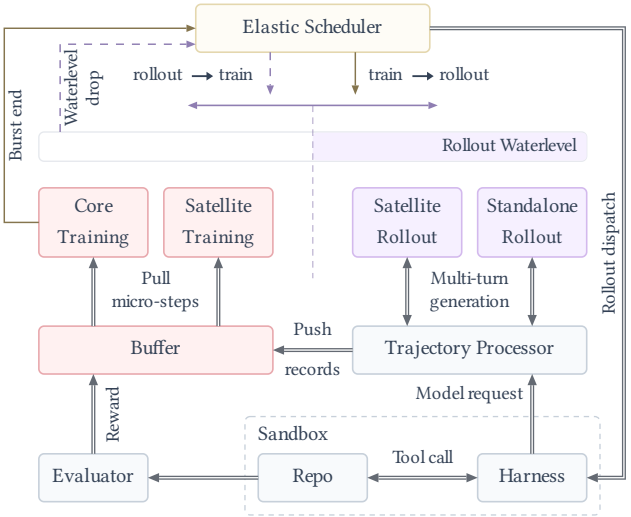


Figure 3: Elastic scheduling and data flow in QWENGYRE. The sandbox hosts the harness and repository, linked by tool calls; Solid and dashed control links denote burst-boundary actions and waterlevel feedback, respectively. Double lines show data flow; the buffer combines trajectory storage and stream packing.

training parallel layout; each can serve rollouts or host a complete training actor replica. The black-box proxy, as part of the trajectory processor forwards model calls to rollout engines and reroutes them during role changes, while harnesses and workspaces remain outside GPU workers. The elastic scheduler dispatches rollout work at startup and burst boundaries and tracks queued and running executions as the rollout waterlevel. As this level falls, cells move to training when the remaining rollout capacity covers outstanding work. The core cell holds the authoritative weights and alone maintains optimizer state. Satellite cells pull its parameter snapshot and can join the ongoing batch before the rollout tail completes.

Trajectory processor. The proxy records exact model-request tokens and behavior log-probabilities in trajectory trees that share common prefixes and preserve each output's original conditioning context. The evaluator scores each execution's preserved workspace, including assessable partial progress after timeout, and sends the reward and its validity status to the buffer. At training admission, QWENGYRE computes group-relative advantages from the original execution rewards and selects at most $J_{\max}$ trajectories per execution by role priority. Selected trajectories inherit their execution's advantage. Only policy-generated tokens are eligible targets, and shared targets contribute once. Averaging losses over selected trainable tokens within each execution, then over executions in the batch, prevents additional trajectories from increasing an execution's aggregate weight.

The two parts connect through a shared stream of training micro-steps, allowing the core to begin a batch before all samples are materialized. Execution termination lowers the scheduling waterlevel; a group becomes ready for training only after all its executions have closed and their designated outcomes are valid. The buffer in Figure 3 stores execution records and associated rewards and packs selected paths from ready groups on demand into micro-steps carrying loss masks and weights. Training cells consume this stream, and the core performs one optimizer update per batch.

Sections 4 and 5 detail the elastic scheduling and trajectory processing planes, respectively.

4 Elastic Scheduler

The elastic scheduling plane coordinates role transitions and streaming training to use available GPUs while serving ongoing rollouts.

4.1 Topology-Aware Resource Organization

Cell layout and roles. QWENGYRE uses Megatron (Shoeybi et al., 2019) for training and SGLang (Zheng et al., 2024) for rollout. Its K *elastic cells* share a training parallel layout; an optional *standalone pool* remains in rollout. We favor small cells for fine-grained resource reallocation, subject to parallelism and topology constraints. Tensor, pipeline, expert, and context parallel groups remain within each cell, with placement favoring high-bandwidth interconnects.

Cells follow a fixed order, with the *core cell* first and *satellite cells* thereafter. The core holds the authoritative weights and is the sole optimizer owner.

Rollout capacity. Rollout capacity is measured as the number of concurrent rollout executions allowed by QWENGYRE. Let C_e and $C_s \geq 0$ denote the capacities of each elastic cell in rollout mode and the standalone pool, respectively, with $C_s = 0$ when the standalone pool is absent. With $k(t)$ cells assigned to training, the available rollout capacity is

$$C_R(t) = C_s + \big(K - k(t)\big)C_e. \tag{3}$$

4.2 Boundary Dispatch and Waterlevel Control

Boundary dispatch. To implement boundary dispatch (Section 2.2), the scheduler tracks three cumulative execution counts: $d(t)$ increases when executions are dispatched into the rollout queue, $p(t)$ increases when they launch, and $f(t)$ increases when they terminate. Thus $d(t) - p(t)$ counts queued executions, $p(t) - f(t)$ counts running executions, and $0 \leq f(t) \leq p(t) \leq d(t)$. Each launched execution increments $f(t)$ exactly once on completion, failure, timeout, or cancellation, including surplus executions under oversampling. Model-request retries remain within the same execution and leave all counters unchanged. Queued entries record the published policy version at dispatch; harnesses and sandboxes are created at launch.

In execution units, the initial dispatch is $d(0) = (\varphi + \mu)NB$, with μNB new executions dispatched after each burst's weight publication. Whenever there is spare rollout capacity (Equation 3) and engines are ready to serve, the scheduler immediately fills available slots by

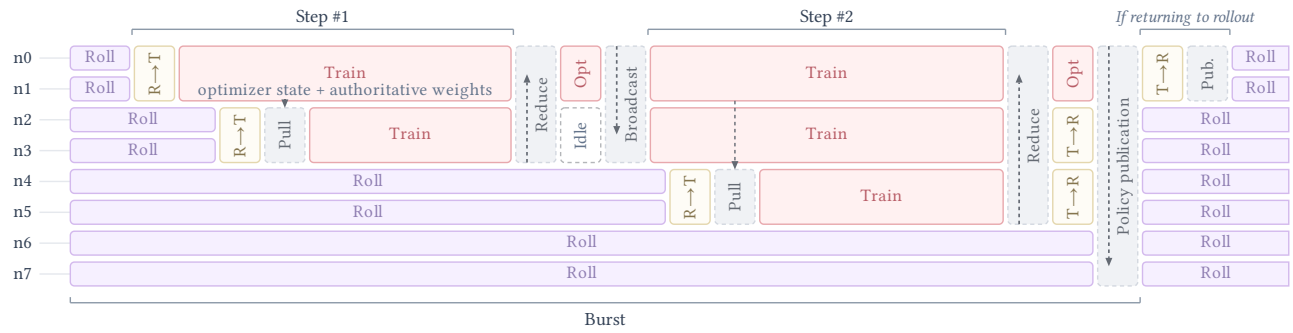


Figure 4: Illustrative timeline of a two-step burst. The core (n0–n1) performs both optimizer updates. Satellite cells join ongoing steps after pulling its current parameters; n6–n7 stay in rollout during this burst. Gradient reduction precedes each update; a parameter broadcast follows the nonfinal update, and rollout policy publication follows the final update. The core's final role transition and local publication (Pub.) occur only when it returns to rollout.

launching

$$\min\{d(t) - p(t),\ C_R(t) - (p(t) - f(t))\} \tag{4}$$

queued executions and leaving any excess queued. The launches in Equation 4 use the existing quota, so $d(t)$ remains fixed within the burst.

Waterlevel control and training supply. The waterlevel $w(t) = d(t) - f(t)$ is nonincreasing within a burst. The next cell can join training only if the remaining rollout capacity covers these outstanding executions:

$$w(t) \le C_R(t) - C_e. \tag{5}$$

Equation 5 is paired with a supply check that counts ready, unassigned groups, capped by the number of groups the current batch still needs. This count must cover at least a fraction $\rho_{\min} \in (0, 1]$ of the batch's B groups continuously for τ before a transition begins. The timer resets after each transition and at each batch start. Cells transition serially, core first and then satellites by index, and remain in training throughout the burst, forming a growing prefix.

Burst-End Weight Sync and Core Retention. After the burst's final optimizer update, all training satellites return to rollout and restore their SGLang engines. The core then publishes the updated weights to all satellite and standalone rollout engines, preparing them for the next burst.

After synchronization, the core remains in training only if at least $\rho_{\min} B$ ready, unassigned groups are available for the next batch and the remaining rollout capacity covers outstanding work plus the pending μNB executions:

$$w(t) + \mu NB \le C_s + (K - 1)C_e. \tag{6}$$

If either the supply check or Equation 6 fails, the core returns to rollout and reshards its parameters into the local SGLang (Figure 4).

4.3 Harness-Preserving Role Transitions

Request rerouting. During a role transition, the proxy reroutes model calls while the harness, workspace, and tool execution state remain in place. The source cell first stops accepting new requests. The scheduler reserves destination capacity for affected executions and updates their routes, then cancels outstanding requests on the source engines. The proxy transparently retries these requests at the destination engines. The scheduler prioritizes available standalone engines, followed by engines in rollout cells in descending cell order. This ordering favors destinations expected to remain available longer, reducing repeated rerouting of the same execution within a burst.

KV-cache migration. KV-cache migration can reduce prefix recomputation during request rerouting. Before a request is canceled on the source engine, the source pins the corresponding cache blocks. Once the request is canceled and cache writes have stopped, the destination can read these blocks through RDMA. The source releases the cache only after the destination acknowledges receipt and takeover.

Role-state management. After request rerouting and cache handoff are complete, the cell offloads rollout parameters and releases KV caches and temporary buffers to reclaim GPU memory, then restores training parameters and buffers. Backend processes and communication groups remain alive across role changes.

4.4 Streaming Training with Dynamic Membership

Centralized dynamic data parallelism. To let satellites join an ongoing batch, the core exposes buffers holding its authoritative weights through Mooncake's Transfer Engine during its transition from rollout to training for the burst's first step, and before each subsequent step. While training on the current batch is still in progress, satellites can join in the predefined cell order. Each satellite registers with the scheduler, pulls the core's weights via RDMA, and begins training without interrupting the core (Figure 4). The weights remain fixed during gradient accumulation, so joining satellites read a consistent snapshot. The controller closes the joining window before the optimizer update.

After the entire batch has been processed, the core aggregates and normalizes gradients from all training cells. It then applies gradient clipping and performs a single optimizer update. After each nonfinal update, it broadcasts the new parameters to the current satellites, which remain in training for the next step.

For K ordered cells, we prebuild $K-1$ sets of communication groups, one for each prefix of $2, \ldots, K$ cells. Within each set, groups connect ranks holding corresponding model shards across cells. Each collective uses the set matching its participating prefix.

Streaming training. Streaming training uses two buffers, shown as the Buffer in Figure 3. The *trajectory buffer* stores execution records and associated rewards by rollout group. The *stream packing buffer* prepares training inputs on demand. When its buffered trajectories run low, it pulls a ready group from the trajectory buffer, prioritizing the oldest policy version. It materializes the group's selected trajectories into token rows and reorders the rows for efficient packing.

The packing buffer emits training inputs in small *micro-steps*, each providing data for a cell's data-parallel ranks. Cells atomically claim micro-steps from the packing buffer's shared queue as compute capacity becomes available, receiving disjoint work. Earlier or faster cells consume more micro-steps, helping participating cells finish the batch at similar times. The packing buffer selects the batch's B groups incrementally and closes the stream only after enqueuing all their micro-steps.

Each cell processes incoming micro-steps with a continuous one-forward-one-backward (1F1B) pipeline. All stages within a cell use the same signal for input availability. When data are temporarily unavailable, the stages complete outstanding communication and pause at consistent scheduling points, preserving pipeline state. After the stream closes and the queue is exhausted, the pipeline drains its remaining work. Each cell that processes micro-steps therefore incurs one startup and one final drain per batch.

5 Trajectory Processor

The trajectory processing plane constructs training samples from harness execution records and task outcomes. During rollout, QWENGYRE preserves the exact model-request tokens and each output's original conditioning context using token-in, token-out (TITO) (Gallouédec & Rasul, 2026) (Section 5.1). After an execution terminates, QWENGYRE evaluates its preserved work, including assessable partial progress after timeout, under the task's scoring rule to obtain an execution-level reward (Section 5.2). At training admission, QWENGYRE selects a bounded set of paths per execution from ready groups by role priority and fixes target masks so shared targets contribute once (Section 5.3).

5.1 TITO and Trajectory Trees

Request records with TITO QWENGYRE implements TITO in the black-box proxy (Section 4.3) to maintain a record of each model call. Each record preserves the exact input and output token IDs, the output tokens' behavior log-probabilities, and associated metadata.

To maintain these records across harness message processing, the proxy also caches original tool-call payloads by call ID. Before matching a subsequent request's history, it restores these payloads while preserving the incoming IDs for tool-response association. This avoids spurious history mismatches caused by tool-call reformatting. The harness keeps its existing message interface, while training consumes the recorded tokens and associated metadata. Each output retains its original conditioning context even if later compaction removes it from the harness's history.

Prefix sharing through trajectory trees QWENGYRE organizes the input/output tokens and associated metadata in the TITO request records described above into a trajectory tree for each execution, following the prefix representation used in black-box agent training (Song et al., 2026a). Each node is built from a request record and stores its incremental input, recorded output, and associated metadata. After the tool-call restoration described above, the framework matches an incoming request's serialized context against existing paths and reuses the tokens stored in the matched nodes. Only new input content is encoded, and the new request record is appended as a node. Diverging contexts create branches; requests with no matching prefix attach to the root. This organization captures the different context paths introduced by compaction and sub-agent calls (Figure 2(c)), while retaining independently generated outputs as separate nodes.

Candidate trajectories remain in this shared representation until selection, avoiding full-history expansion for every path. Selected paths are materialized from the stored token segments and their aligned metadata.

5.2 Partial Scoring

Scoring preserved work At execution closure, QWENGYRE fixes the committed policy-generation request record and identifies any available artifact state to assess. QWENGYRE evaluates the preserved work under the task's scoring rule after an execution terminates, including after timeout. An unfinished execution can receive a valid score when its completed work is assessable. For these assessed outcomes, the data plane does not derive rewards from elapsed time, trajectory length, or termination status. Using executable checks or task-specific agentic evaluation (Zhuge et al., 2024), the evaluator returns a score, its validity status, and a reference to the assessed state.

Preserving the scope of an assessment An XLong execution may evaluate intermediate artifacts, revise them, and abandon branches. Each assessment remains associated with its execution, artifact version, and available branch reference; later assessments do not overwrite these associations. A critique used by a subsequent policy request remains part of its input, but its evaluator provenance excludes it from policy training targets. The

designated task assessment supplies one reward for each scored execution; intermediate assessments remain associated with the states they evaluate.

Handling failures An evaluator failure leaves the assessment unresolved; evaluation may be retried against the same preserved state within the execution's overall time budget. If this budget expires without a valid reward, the recorded trajectories are excluded from training. An unresolved assessment is distinct from a valid zero score, which remains usable for training.

For rare complete failures that leave no assessable work, such as image-pull failures, QWENGYRE inserts a dummy trajectory with zero reward as a failure placeholder.

5.3 Trajectory Sampling

The trajectory tree records all context in the execution, which QWENGYRE turns into training data in two steps: it first masks every token that should not be trained, and then converts the tree into individual training trajectories.

Trainable tokens are defined by provenance. A node's tokens are eligible training targets only if the policy produced them during rollout. TITO (Section 5.1) records whether each token was emitted by our SGLang engines or supplied from outside. System prompts, user prompts, tool results, and assistant-role content the harness injects rather than samples from the policy will be masked and carry no training signal: they are retained as conditioning context only.

Sampling trajectories from the tree. Candidate trajectories correspond to the leaves of the trajectory tree: a leaf together with the prefix path from the root defines one trajectory. At XLong-horizons a single execution can produce many candidate trajectories, and their number may vary widely across executions. Simply retaining all trajectories may lead to a decrease in training efficiency, so QWENGYRE retains at most $J_{\max}$ training trajectories per execution and fills this budget in order of expected importance. To rank the candidates, QWENGYRE classifies each leaf by the role of the request sequence it terminates. For a Claude Code harness these classes are, for the main agent and for each sub-agent, its task-driving trajectory and its summary trajectory, ordered by the priority

$$\begin{aligned} &\text{main} > \text{main-summary} \\ &\quad > \text{sub-agent} > \text{sub-agent-summary}. \end{aligned} \tag{7}$$

The ordering in Equation 7 follows two principles. The main agent's turns are what the task-level reward most directly credits, whereas sub-agent trajectories contribute only through delegated subtasks; and within each agent, turns that advance the task rank above the summary trajectory, which manages context rather than acting on the task.

Admission starts from the provenance mask above and draws trajectories one at a time. Each step takes the highest-priority class that still has a leaf with unmasked trainable tokens, and draws one leaf from it with probability proportional to its unmasked count. The drawn trajectory's targets are fixed to the tokens still unmasked at this draw; QWENGYRE then masks its entire root-to-leaf path in the tree. This keeps a shared prefix from being trained more than once and leaves already-fixed targets untouched, while the lowered counts reweight the remaining leaves for the next draw. Admission repeats until no unmasked trainable token remains or $J_{\max}$ trajectories are admitted, yielding $J_i \leq J_{\max}$ trajectories from execution i (Figure 2), each inheriting the execution's group-relative advantage.

The admitted trajectories retain their execution identities, recorded contexts and behavior log-probabilities, and fixed target masks. Materialization and packing preserve these annotations for the execution-level objective.

6 Experimental Results

6.1 Experimental Setup

Tasks. We evaluate QWENGYRE on three agentic workloads: NL2RepoBench (Ding et al., 2026), DeepSWE (Huang et al., 2026), and TerminalBench (Merrill et al., 2026). We construct training data internally for all three workloads. For benchmark evaluation, we follow each benchmark's prescribed evaluation protocol. NL2RepoBench covers the XLong setting of building software repositories from natural-language specifications; during training, we use a separate Qwen 3.7-Max evaluator to score the generated code using independently written tests. DeepSWE covers software engineering in existing repositories; during training, we use repository tests for evaluation. TerminalBench comprises multi-turn terminal tasks; during training, we use task-specific verifiers for evaluation. All workloads use Claude Code 2.1.220 (Anthropic, 2026) to execute shell commands and manipulate files in containerized environments.

Models. We use Qwen 3.6 122B and Qwen 3.8 2.4T. For Qwen 3.6 122B, the NL2RepoBench experiments use 32 nodes, organized into eight cells of four nodes each by default, with an inference concurrency limit of 144 per cell. The cell-granularity ablation also evaluates four 8-node cells under the same 32-node budget. DeepSWE and TerminalBench follow the same protocol, using 24 nodes organized into six cells with an inference concurrency limit of 192 per cell. For Qwen 3.8 2.4T, the NL2RepoBench traces use four cells of 48 nodes each, with a total inference concurrency limit of 384. Unless otherwise stated, we use no dedicated standalone rollout nodes.

RL algorithm. All methods use our adaptation of Group Sequence Policy Optimization (GSPO) (Zheng et al., 2025), combining token-level importance weighting with execution-level clipping. Token importance weights are capped at 5, and the sequence-level clipping bounds are $[0.995, 1.005]$. Each group contains 16 independent rollout executions; each update consumes 32

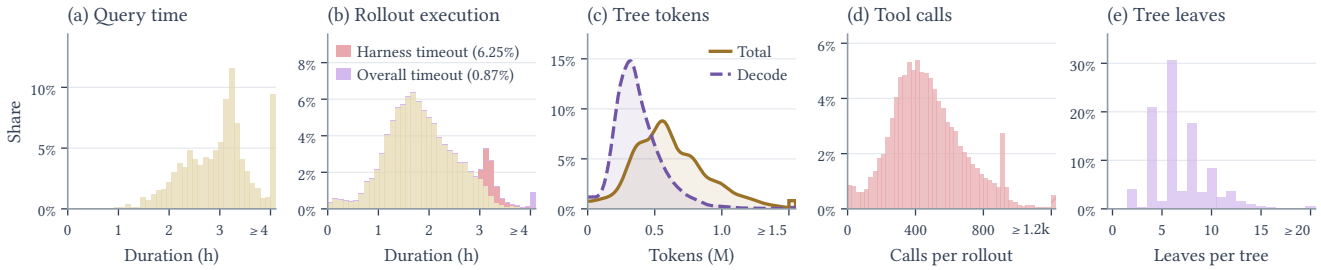


Figure 5: NL2RepoBench execution distributions with Qwen 3.6 122B at $\mathbb{E}[d] = 1.5$ over steps 1–48. (a) Query durations, measured as the longest of each query's 16 rollout execution durations. (b) Rollout execution durations. (c) Total (solid) counts input and output tokens per rollout tree; Decode (dashed) counts generated output tokens. Both count shared prefixes once. The smoothed curves show shares per 50k tokens. (d) Tool calls per rollout execution. (e) Leaves per rollout tree. In (b), pink and purple mark harness and overall timeouts, respectively; executions with both flags appear only in purple. Final bins labeled $\geq$ collect values at or above the indicated thresholds.

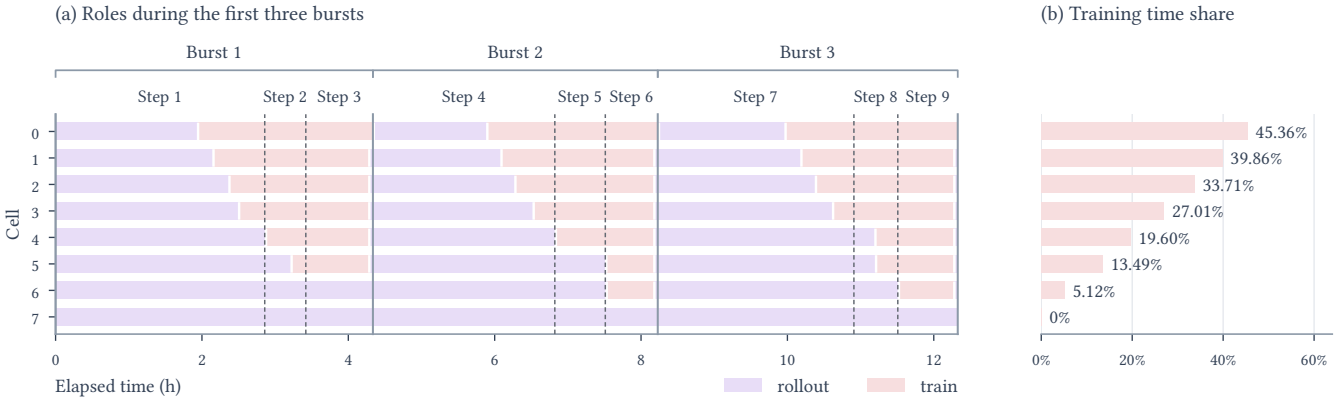


Figure 6: Execution profile of QWENGYRE on NL2RepoBench with Qwen 3.6 122B at $\mathbb{E}[d] = 1.5$. (a) Cell roles during the first three bursts (steps 1–9). (b) Each cell's recorded training time as a fraction of the full observation window. Cell 7 remains in rollout throughout this interval.

groups for NL2RepoBench or 48 for DeepSWE and TerminalBench. The default trajectory cap is $J_{\max} = 5$ per rollout execution. All methods use Adam (Kingma & Ba, 2015) with a constant learning rate of 10^{-6}.

Baselines. We compare QWENGYRE with two scheduling baselines. *Async* assigns separate, fixed GPU pools to rollout and training, allowing training steps to consume ready rollout groups while other executions continue. It uses boundary dispatch with a target queue depth, refilling consumed groups' slots after weight publication. *Colocate* alternates a shared GPU pool between rollout and training. Once the admitted rollout executions finish, the entire pool switches to training and processes the completed rollout groups in a burst of consecutive training steps.

We compare methods at common target values of mean scheduling staleness $\mathbb{E}[d]$, defined in Section 2.2. All three methods use boundary dispatch; we choose the extra-dispatch parameter φ and the number of training steps per burst μ using Equation 2. The original Colocate cannot dispatch extra groups without interrupting the harness, so it uses $\varphi = 0$. For example, at $\mathbb{E}[d] = 1.5$, Async can use $\mu = 1$ with $\varphi = 1.5$, Colocate uses $\mu = 4$ with $\varphi = 0$, and QWENGYRE can use $\mu = 3$ with $\varphi = 0.5$.

6.2 End-to-End Performance

NL2RepoBench on Qwen 3.6 122B Figure 5 shows mean durations of 1.93 hours per rollout execution and 2.96 hours per query; 9.51% of queries take at least four hours. Figure 5(b) shows harness and overall timeout rates of 6.25% and 0.87%, respectively. Harness timeouts indicate that task execution has timed out; the results are still scored and used for training. Failed evaluations can be retried within the overall execution time budget. An overall timeout exhausts this budget before a valid reward is obtained, so the execution is discarded from training.

Figure 7(a–d) shows that QWENGYRE achieves 1.42–1.53× speedups over Async and 1.36–1.47× over Colocate after 48 training steps at both mean scheduling staleness targets. Figure 7(e,f) shows comparable training-score curves across methods, indicating that faster training is achieved while maintaining task performance.

The role timeline in Figure 6(a) shows how QWENGYRE dynamically allocates resources between training and inference. Cells enter training at different points while other cells remain in rollout. Figure 6(b) shows how training time shares vary across cells over the full observation window. This staggered allocation allows training to begin with a subset of cells and subsequently expand.

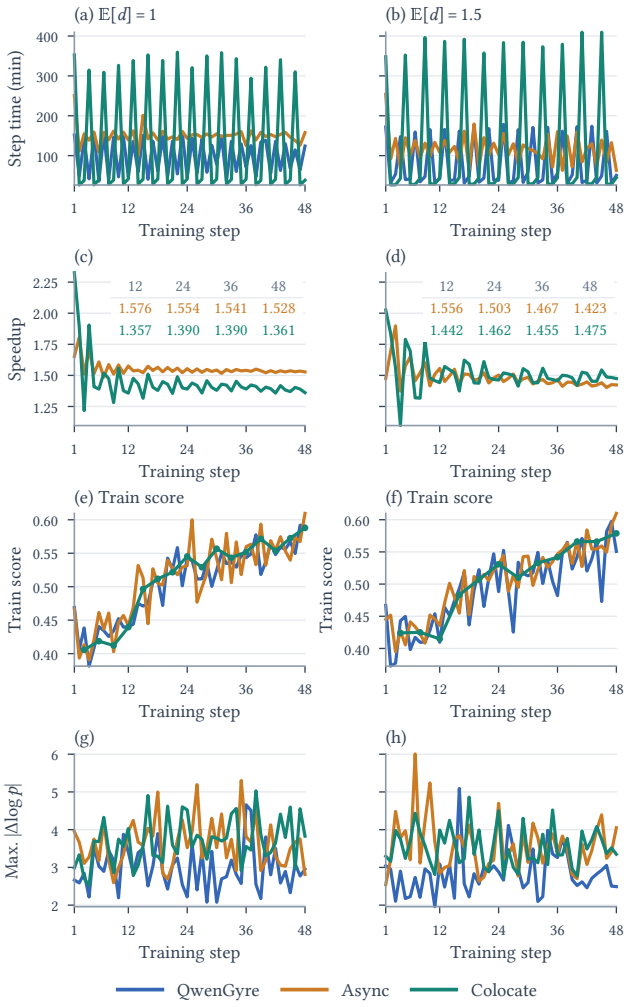


Figure 7: NL2RepoBench results with Qwen 3.6 122B at $\mathbb{E}[d] = 1$ (left) and $\mathbb{E}[d] = 1.5$ (right). Speedup is cumulative baseline time divided by cumulative QwenGyre time. In (c,d), in-plot tables give speedups at selected checkpoints: Async in the upper row and Colocate in the lower row, with matching curve colors. Max. $|\Delta \log p|$ is the maximum absolute difference between current-policy and recorded rollout-policy log-probabilities over unmasked response tokens.

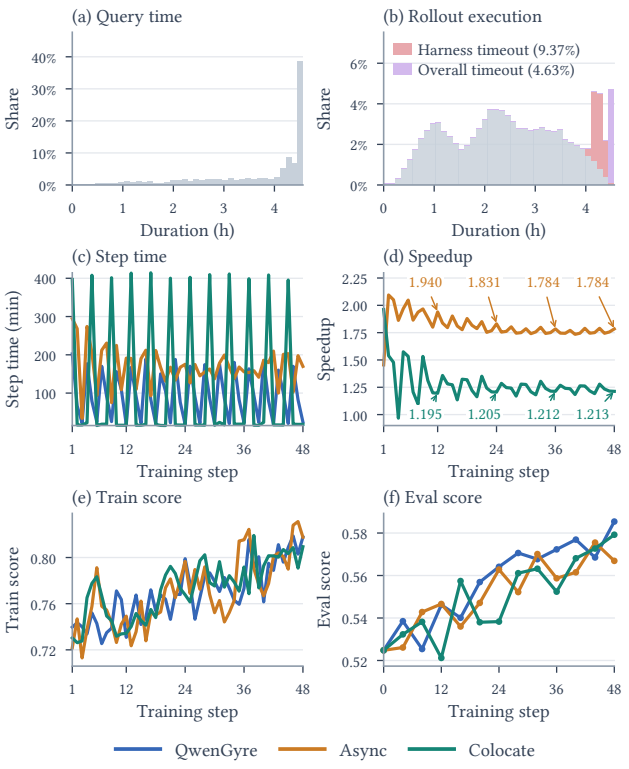


Figure 8: NL2RepoBench with Qwen 3.8 2.4T. (a) Measured query durations, using the longest rollout execution wall time per query. (b) Measured rollout execution durations. The final bin collects durations ≥ 4.45 h. In (b), pink and purple mark harness and overall timeouts, respectively. (c,d) Step times and cumulative speedups over 48 steps. Arrows in (d) label speedups at selected checkpoints in matching colors. (e,f) Training and evaluation scores.

It uses resources that a fixed Async partition leaves assigned to rollout and avoids requiring Colocate's pool-wide phase transition before training can proceed. The average switching times are 8.52 s for rollout to training and 3.46 s for training to rollout. Both are negligible compared with rollout executions lasting hours.

NL2RepoBench on Qwen 3.8 2.4T On NL2RepoBench, Qwen 3.8 2.4T has a higher timeout rate than Qwen 3.6 122B, even though we extend the timeout limit. Figure 8(a,b) shows the query and rollout execution duration distributions. Among queries with complete duration records, 61.1% take at least four hours, compared with 9.51% for Qwen 3.6 122B. In 38.2% of these queries, the longest rollout execution terminates with an overall timeout.

As shown in Figure 8(c,d), QwenGyre completes 48 training steps in 75.42 hours, compared with 134.55 hours for Async and 91.47 hours for Colocate, yielding end-to-end speedups of 1.78× and 1.21×, respectively. The concentration of query durations near the timeout limit helps explain the difference in gains over the two baselines. It suggests fewer opportunities for QwenGyre to reclaim cells early and expand training gradually, consistent with the smaller gain over Colocate, which also shares its resource pool between training and rollout. Async's fixed partition leaves resources reserved for training unavailable to rollout, even when training is idle. By dynamically allocating these resources to rollout, QwenGyre can increase inference capacity during long executions, yielding a larger gain over Async.

Figure 8(e) shows that QwenGyre's training scores match those of Async and Colocate. Its evaluation pass-rate improves from 52.48% to 58.54% over 48 training steps (Figure 8(f)).

DeepSWE and TerminalBench on Qwen 3.6 122B DeepSWE and TerminalBench share the same end-to-end evaluation protocol: Qwen 3.6 122B, 24 training steps, and all three methods (QwenGyre, Async, and Colocate) at target mean scheduling staleness values $\mathbb{E}[d] = 1$ and 1.5. Table 2 reports the resulting cumulative speedups.

Across the reported DeepSWE and TerminalBench con-

Table 1: Scheduling ablations over the first 12 NL2RepoBench training steps with Qwen 3.6 122B at $\mathbb{E}[d] = 1.5$. A0, C0, and E0 reuse the Async, Colocate, and QWENGYRE base configurations from Section 6.2. Arrows show the evolution paths; E1 and E2 branch independently from E0. Switch is – for fixed partitions, Global for switching all nodes together, Coarse for switching the Colocate pool together, and Fine for switching cells independently. Standalone nodes remain in rollout. Allocation lists training/rollout (T/R) or Colocate/standalone (C/S) node counts; cell entries give cells × nodes per cell. Total is the time for all 12 steps in hours; Time / E0 normalizes it to E0. Lower is better.

ID	Evolution	Stream	Switch	(φ, μ)	Allocation	Total (h)	Time / E0
A0	Original Async	Disabled	–	$(1.5, 1)$	16 T / 16 R	25.08	1.556
A1	A0 with adjusted partition	Disabled	–	$(1.5, 1)$	8 T / 24 R	27.01	1.676
A2	A0 + multi-step burst	Disabled	–	$(0.5, 3)$	16 T / 16 R	32.58	2.021
A3	A0 + streaming	Enabled	–	$(1.5, 1)$	16 T / 16 R	22.18	1.376
C0	Original Colocate	Disabled	Global	$(0, 4)$	32 C / 0 S	23.23	1.442
C1	C0 + standalone rollout nodes	Disabled	Coarse	$(0.5, 3)$	28 C / 4 S	22.56	1.400
C2	A3 + role switching; C1 + streaming	Enabled	Coarse	$(0.5, 3)$	28 C / 4 S	19.23	1.193
C3	C2 with adjusted allocation	Enabled	Coarse	$(0.5, 3)$	16 C / 16 S	20.69	1.284
E0	C2 + fine-grained elastic allocation	Enabled	Fine	$(0.5, 3)$	8 cells × 4	16.12	1.000
E1	E0 with one step per burst	Enabled	Fine	$(1.5, 1)$	8 cells × 4	17.87	1.109
E2	E0 with four 8-node cells	Enabled	Fine	$(0.5, 3)$	4 cells × 8	16.47	1.022

Table 2: 24-step speedup of QWENGYRE over each baseline with Qwen 3.6 122B. The (φ, μ) column gives QWENGYRE's extra dispatch in batch units and training steps per burst. Async uses $(\varphi, \mu) = (\mathbb{E}[d], 1)$; Colocate uses $(\varphi, \mu) = (0, 2\mathbb{E}[d] + 1)$, following Equation 2.

Dataset	$\mathbb{E}[d]$	(φ, μ)	vs. Async	vs. Colocate
DeepSWE	1	$(0.5, 2)$	1.569	1.610
DeepSWE	1.5	$(0.5, 3)$	1.572	1.824
TerminalBench	1	$(0.5, 2)$	1.383	1.809
TerminalBench	1.5	$(0.5, 3)$	1.430	1.849

figurations, QWENGYRE provides consistent end-to-end speedups over both Async and Colocate.

6.3 Ablation Study

Table 1 traces two paths toward QWENGYRE: adding streaming training to Async and adding standalone rollout capacity to Colocate.

Adding streaming to Async. Reallocating nodes from training to rollout (A0 to A1) increases end-to-end time by 7.7%. Although training nodes wait for rollout data, executions finish in bursts; the smaller training pool consumes these bursts more slowly, delaying weight publication and replacement rollouts under boundary dispatch. Longer training bursts (A2) increase time by 29.9% relative to A0 because the fixed rollout pool cannot supply them efficiently. Streaming (A3) instead reduces time by 11.6% by starting training on completed queries before a full batch is ready. However, streaming captures only part of the potential gain. The fundamental limitation remains Async's fixed resource partition: it cannot reallocate idle GPUs between training and rollout as their demands change.

Adding standalone rollout nodes to Colocate. while the colocated pool switches to training, preserving harness state. Although extra dispatch mitigates the impact of rollout stragglers, fewer training steps are completed per burst, limiting the end-to-end improvement over C0 to just 2.9%.

Adding streaming to C1 (C2) reduces time by 14.8%. However, all 28 colocated nodes must still switch together, so streaming training can begin only late in the burst. Increasing standalone capacity (C3) shrinks the pool that must switch together, but also leaves fewer nodes for training. The resulting longer training phase delays burst completion and the next dispatch, increasing end-to-end time by 7.6% relative to C2. The fundamental limitation is the coarse switching granularity, which prevents training from fully using the idle capacity that emerges as colocated rollouts finish at different times.

Combining streaming and fine-grained allocation. E0 builds on C2 by replacing its fixed split of 28 colocated nodes and four standalone rollout nodes with eight independently switching 4-node cells, while retaining streaming and three training steps per burst. Training can thus start on a subset of cells and expand as data arrives while other cells continue rollout, yielding a 1.19× speedup over C2.

E1 retains E0's streaming and cell allocation but uses one training step per burst, with extra dispatch increased from 0.5 to 1.5 to preserve scheduling staleness. Although E1 takes 10.9% longer than E0, it still achieves a 1.24× speedup over A3, which also uses streaming and one training step per burst, and a 1.08× speedup over C2. These results show that fine- grained allocation provides substantial gains even without multi-step bursts; longer bursts offer an additional benefit.

Streaming makes rollout data available for training earlier, while fine-grained allocation supplies training resources as that data arrives.

Cell granularity. E2 builds on E0 by regrouping the

same 32 nodes from eight 4-node cells into four 8-node cells, retaining streaming and $(\varphi, \mu) = (0.5, 3)$. It completes the 12 training steps in 16 hours 28 minutes (16.47 hours), only 2.2% longer than E0's 16.12 hours. In E0, rollout fully hides training, with 20.85% slack remaining. E2's coarser cell granularity exposes a small amount of training on the critical path, while most training remains overlapped with rollout. This explains the modest impact of reducing the number of cells from eight to four in this setting.

Trajectory sampling budget. Sampling only the main trajectory per execution is insufficient: it yields lower training scores and larger gradient norms over the first 42 training steps (Figure 9). Increasing the budget (Section 5.3) to five trajectories matches the scores obtained by sampling all available trajectories, with similarly small gradient norms. Over the same window, mean forward–backward time per step with caps of one and five is 41.4% and 74.8% of the uncapped baseline, respectively. A cap of five therefore preserves comparable training scores while reducing forward–backward time by 25.2%.

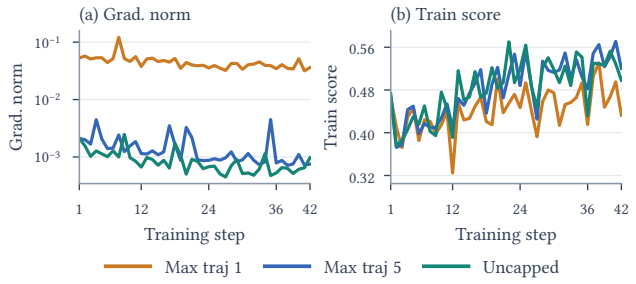


Figure 9: Trajectory-budget ablation for QWENGYRE on NL2RepoBench with Qwen 3.6 122B over training steps 1–42. (a) Gradient norm on a logarithmic scale. (b) Mean training score. Max traj 1 samples only the main trajectory per execution; Max traj 5 admits up to five trajectories; "Uncapped" uses all available leaf trajectories.

7 Related Work

Agentic RL execution. General-purpose LLM RL frameworks such as HybridFlow/veRL, OpenRLHF, ReaL, and ROLL compose rollout, reference, reward, and policy-update workers under different placement strategies (Sheng et al., 2025; Hu et al., 2025; Mei et al., 2025; Wang et al., 2025b). Agent-oriented runtimes extend these abstractions with multi-turn tools, containerized environments, asynchronous dispatch, and trajectory-level scheduling (Jiang et al., 2026; Wang et al., 2025c; Zhang et al., 2025; Cao et al., 2025; Zhang et al., 2026a;b).

Synchronous RL frameworks preserve explicit policy-update boundaries but expose rollout-length skew. Recent work mitigates this tail through scheduling, length-aware packing, partial-rollout continuation, and tail isolation (Qin et al., 2026; Gao et al., 2026a; Zhou et al., 2025; Xu et al., 2026b). Asynchronous RL frameworks instead stream trajectories between disaggregated rollout and trainer pools, using bounded staleness, multiple policy versions, or fixed-weight trajectory groups to control training semantics (Fu et al., 2025; Han et al., 2025; Li et al., 2026; Hu et al., 2026; Chen et al., 2026b). SAO and SPO++ further adapt policy optimization to asynchronously arriving agent trajectories (Hou et al., 2026; Ruan et al., 2026).

Elastic rollout–training scheduling. Recent RL frameworks dynamically adjust the resource boundary between rollout and training. TideRL adapts rollout and reference execution using readiness signals (Ren et al., 2026b); BiDiRL allows either side of an asynchronous pipeline to borrow idle resources from the other (Tan et al., 2026); and DynaResize reallocates GPUs between rollout and training pools using communicator reuse and state staging (Du et al., 2026a). Libra combines a global resource planner with an elastic hybrid pool whose workers can join training as additional data-parallel replicas (Chen et al., 2026a). Other RL frameworks obtain elasticity from complementary phases across RL jobs, online-serving clusters, or preemptible rollout resources (Wu et al., 2026a; Gao et al., 2026b; Wu et al., 2026b), while AReaL-DTE reduces cross-role policy synchronization through sparse weight transfer (Peng et al., 2026).

Black-box harnesses and structured trajectories. Harness-native RL frameworks connect opaque agent runtimes to policy optimization by instrumenting or proxying model calls. Agent Lightning decouples agent execution from training, while Polar and LEGO-RL preserve rollout-time token identity under harness-controlled context processing (Luo et al., 2025b; He et al., 2026b; Xu et al., 2026a; Du et al., 2026b). ClawGym II reconstructs captured requests as shared-prefix trees and adapts PPO and GRPO so that each shared node contributes training signal only once (Song et al., 2026a); its retained paths share a reward derived from the final workspace evaluation.

Related work exploits execution structure at other layers. BPO, IAPO, and MileGPO use branches, dependency graphs, or intermediate milestones for finer-grained credit assignment (He et al., 2026a; Ren et al., 2026a; Qian et al., 2026), whereas psRL and related frameworks reuse shared prefixes after sample construction (Yu et al., 2026; Wang et al., 2025a). These techniques address trajectory capture, credit assignment, or prefix computation individually. QWENGYRE instead couples readiness-driven actor reconfiguration with a trajectory processor preserving role, branch, environment-state, and evaluator provenance throughout XLong rollout collection, admission, and training materialization.

8 Limitations

GPU budget and cell granularity. Elastic scheduling needs multiple independently switchable cells while maintaining enough rollout capacity to serve unfinished executions. Each cell must accommodate the model's memory and training parallelism requirements, so a very small GPU budget may not support this organi-

zation. Such deployments are generally impractical for the resource-intensive XLong workloads targeted here. Within our evaluated regime, a modest number of cells is sufficient: under the same 32-node budget, regrouping eight 4-node cells (E0) into four 8-node cells (E2) increases execution time by only 2.2% (Table 1). E0 fully hides training behind rollout, while E2 exposes a small amount of training on the critical path. Most of the overlap benefit is therefore retained with four cells in this setting. This result concerns cell granularity at a fixed budget; efficiency under substantially smaller total GPU budgets remains unevaluated.

Shuffling with streaming training. With multiple training steps per burst ($\mu > 1$), streaming training assigns ready rollout groups incrementally to successive updates. The current design therefore cannot globally shuffle data across all minibatches before the burst starts. Supporting such a shuffle would require waiting for the complete burst's data before its first update, delaying training and sacrificing some overlap with rollout. This constrains training procedures that depend on globally randomized minibatch assignment. Single-step bursts ($\mu = 1$) avoid the cross-minibatch split while retaining elastic allocation and streaming. At matched scheduling staleness, E1 takes 10.9% longer than E0, but still achieves a 1.24× speedup over streaming Async (A3) and a 1.08× speedup over Colocate with streaming (C2) (Table 1). Thus, multi-step bursts provide an additional efficiency gain, while single-step bursts remain a practical option. These ablations quantify execution time; the effect of sample ordering on learning quality has not been isolated.

9 Conclusion

We presented QWENGYRE, which combines elastic scheduling and trajectory processing for online RL through unmodified harnesses at XLong-horizons. It reallocates GPUs while preserving live executions and constructs bounded training samples with original contexts and balanced execution-level contributions. We demonstrate these capabilities through XLong RL training of our flagship model, Qwen 3.8 2.4T. Across three workloads, QWENGYRE achieves up to 1.78× speedup over Async and 1.85× over Colocate under equal GPU budgets and matched scheduling staleness, with comparable training scores.

References

Anthropic. Claude Code. Official software repository, 2026. URL https://github.com/anthropics/claude-code.

Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R. Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. SkyRL-Agent: Efficient RL training for multi-turn LLM agent, 2025. URL https://arxiv.org/abs/2511.16108.

Kaiwen Chen, Xin Tan, Jingzong Li, and Hong Xu. Libra: Efficient resource management for agentic RL post-training, 2026a. URL https://arxiv.org/abs/2606.03077.

Rongjian Chen, Jianmin Hu, Kejiang Ye, and Minxian Xu. RolloutPipe: Overlapping pipelined rollout and training in disaggregated on-policy LLM reinforcement learning. In *Proceedings of the 2026 International Conference on Cognitive Computing (ICCC 2026)*, 2026b. URL https://arxiv.org/abs/2606.26997.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*. PMLR, 2026. URL https://icml.cc/virtual/2026/poster/61047.

Rishi Desai, Jesse Hu, Joan Cabezas, Neel Harsola, Pratyush Shukla, Roey Ben Chaim, Adnan El Assadi, Omkaar Mukund Kamath, Fenil Faldu, Prannay Hebbar, Jiankai Sun, Yiyuan Li, Pramod Srinivasan, Ishan Gupta, Christopher Settles, Daniel Wang, Derek Chen, Pranav Raja, Albert Liu, Marek Šuppa, Nevasini Sasikumar, Luyang Kong, Erik Quintanilla, Xiangyi Li, Ivan Bercovich, and Steven Dillmann. SWE-Marathon: Can agents autonomously complete ultra-long-horizon software work?, 2026. URL https://arxiv.org/abs/2606.07682.

Jingzhe Ding, Shengda Long, Changxin Pu, Ge Zhang, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Minghao Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Haoran Wang, Tom Tang, Weihao Xie, Yishuo Yuan, Jiayu Zhang, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Liam Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, and Wenhao Huang. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*. PMLR, 2026. URL https://icml.cc/virtual/2026/poster/60772.

Hanlin Du, Zhiyuan Yan, Yungang Bao, and Sa Wang. DynaResize: Runtime GPU reallocation for disaggregated LLM post-training, 2026a. URL https://arxiv.org/abs/2607.22614.

Yiming Du, Yuxin Jiang, Tao Yuan, Jianbo Dai, Shaowei Wang, Jierun Chen, Chaofan Tao, Xianzhi Yu, Lifeng Shang, Kam-Fai Wong, Xiaohui Li, and Haoli Bai. LEGO-RL: Harness-native reinforcement learning for coding agents, 2026b. URL https://arxiv.org/abs/2608.17393.

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. In *Advances in Neural Information Processing Systems*, volume 38, pp. 36256–36282. Curran Associates, Inc., 2025. doi: 10.52202/085713-1218. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/33c00862bfa29ac72ecf630a41e19352-Abstract-Conference.html.

Quentin Gallouédec and Kashif Rasul. Agentic RL: Token-in, token-out done right. Hugging Face Blog, 2026. URL https://huggingface.co/blog/huggingface/tito.

Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. RollPacker: Taming long-tail rollouts for RL post-training with tail batching. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pp. 849–866, Renton, WA, 2026a. USENIX Association. ISBN 978-1-939133-54-0. URL https://www.usenix.org/conference/nsdi26/presentation/gao-wei.

Wei Gao, Yuheng Zhao, Dilxat Muhtar, Dakai An, Xuchun Shang, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Weixun Wang, Ju Huang, Teng Ma, Siran Yang, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. ROSE: Rollout on serving GPUs via cooperative elasticity for agentic RL, 2026b. URL https://arxiv.org/abs/2605.06534.

Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, Sergei Skvortsov, and Boris Yangel. Training long-context, multi-turn software engineering agents with reinforcement learning, 2025. URL https://arxiv.org/abs/2508.03501.

Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, Huazhong Ji, Wenjie Liu, Yu Huang, Yixiang Zhang, Chenyi Pan, Jing Wang, Xin Huang, Chunsheng Li, and Jianping Wu. AsyncFlow: An asynchronous streaming RL framework for efficient LLM post-training, 2025. URL https://arxiv.org/abs/2507.01663.

Bowei He, Yankai Chen, Xiaokun Zhang, and Xue Liu. Branching policy optimization: Sandbox-native language agent reinforcement learning. In *World Artificial Intelligence Conference Academic (WAICA)*, 2026a. URL https://arxiv.org/abs/2607.14171. Main Conference.

Zhiyuan He, Siwei Zhang, Zhiwen Zhou, Yuqing Yang, Yu Kang, Yuge Zhang, Luna K. Qiu, Tin Yan Tsui, Jiahang Xu, and Chong Luo. Agent Lightning v1.0: Towards harnessed agentic RL, 2026b. URL https://arxiv.org/abs/2608.17528.

Deyao Hong, Yizhe Chi, Wenyi Li, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. SWE Refactor Bench: Can coding agents complete a long-horizon, whole-repository stack migration?, 2026. URL https://arxiv.org/abs/2608.23564.

Zhenyu Hou, Yujiang Li, Jie Tang, and Yuxiao Dong. Single-rollout asynchronous optimization for agentic reinforcement learning, 2026. URL https://arxiv.org/abs/2607.07508.

Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, Wenkai Fang, Xianyu, Yu Cao, Haotian Xu, and Yiming Liu. OpenRLHF: A ray-based easy-to-use, scalable and high-performance RLHF framework. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 656–666, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-demos.48. URL https://aclanthology.org/2025.emnlp-demos.48/.

Tianhao Hu, Xiangcheng Liu, Yuchun Miao, Youshao Xiao, Hongyu Zang, Yang Zheng, Xuan Huang, Jinrui Ding, Yufei Zhang, Yu Yang, Yi-Kai Zhang, Yueqing Sun, Chengcheng Han, Xiandi Ma, Wei Wang, Qi Gu, Yerui Sun, Yuchen Xie, and Xunliang Cai. DORA: A scalable asynchronous reinforcement learning system for language model training, 2026. URL https://arxiv.org/abs/2604.26256.

Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026. URL https://arxiv.org/abs/2607.07946.

Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, Tianyu Pang, and Wenhu Chen. VerlTool: Towards holistic agentic reinforcement learning with tool use. *Transactions on Machine Learning Research*, 2026. URL https://arxiv.org/abs/2509.01055.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, pp. 54107–54157, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acfe6cbe9ca88fb84-Paper-Conference.pdf.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *The Third International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

Haoyang Li, Sheng Lin, Fangcheng Fu, Yuming Zhou, Xiaodong Ji, Yanfeng Zhao, Lefeng Wang, Jie Jiang, and Bin Cui. StaleFlow: Staleness-aware data management for mitigating data skewness in fully disaggregated RL post-training, 2026. URL https://arxiv.org/abs/2601.12784. To appear at ACM SIGMOD 2027.

Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpay Ariyak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. DeepSWE: Training a fully open-sourced, state-of-the-art coding agent by scaling RL. Together AI research blog, July 2025a. URL https://www.together.ai/blog/deepswe.

Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent Lightning: Train ANY AI agents with reinforcement learning, 2025b. URL https://arxiv.org/abs/2508.03680.

Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. ReaL: Efficient RLHF training of large language models with parameter reallocation. In *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/3b3889d313ba9476c12c2d77ea66b24f-Abstract-Conference.html.

Mike Merrill, Alexander Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong (Ryan) Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Rytting, Ryan Marten, Yixin Wang, Jenia Jitsev, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations*, volume 2026, pp. 40903–40986, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/444a3737adaee10d86ad2ef5f74468e6-Paper-Conference.pdf.

Yingqi Peng, Jiawei Zhang, Wenhao Zhou, Ruida Xu, Ran Yan, Wei Dong, Yi Gao, Zhiqiang Ding, Tongkai Yang, and Binhang Yuan. AReaL-DTE: Sparse policy-weight transfer for online agentic reinforcement learning, 2026. URL https://arxiv.org/abs/2608.00455.

Bo Qian, Yuting Wu, Shuang Zeng, Huaiyu Wan, Dalin Zhang, and Jiqiang Liu. MileGPO: Milestone inference with local evidence for graph-based policy optimization of long-horizon LLM agents, 2026. URL https://arxiv.org/abs/2608.19803.

Ruoyu Qin, Weiran He, Weixiao Huang, Yangkun Zhang, Yikai Zhao, Bo Pang, Xinran Xu, Yingdi Shan, Yongwei Wu, and Mingxing Zhang. Seer: Online context learning for fast synchronous LLM reinforcement learning. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pp. 883–901, Seattle, WA, 2026. USENIX Association. ISBN 978-1-939133-55-7. URL https://www.usenix.org/conference/osdi26/presentation/qin.

Bo Ren, Yirong Mao, Yi Yang, and Wenhui Que. IAPO: Influence-aware policy optimization for credit assignment in multi-turn service agents, 2026a. URL https://arxiv.org/abs/2608.24588.

Yanyu Ren, Xizheng Wang, Xiao Liu, Bowen Lv, Hanchen Zhang, Shudan Zhang, Hanyu Lai, Shuai Wang, Li Chen, Dan Li, and Jie Tang. TideRL: Boosting agentic RL goodput with readiness-aware scheduling, 2026b. URL https://arxiv.org/abs/2608.10402.

Kai Ruan, Jinghao Lin, Qianshan Wei, Ziqi Zhou, and Zihe Huang. SPO++: Stream-aligned policy optimization for asynchronous agentic RL, 2026. URL https://arxiv.org/abs/2608.24870.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297. ACM, 2025. doi: 10.1145/3689031.3696075. URL https://doi.org/10.1145/3689031.3696075.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism, 2019. URL https://arxiv.org/abs/1909.08053.

Huatong Song, Fei Bai, Ming Yang, Renyuan Li, Jia Deng, Jujie He, Zhange Zhang, Daixuan Cheng, Yan Xing, Qi Yun, Xuxing Chen, Danyang Li, Feng Chang, Chuan Hao, Ran Tao, Jian Yang, Bryan Dai, Wayne Xin Zhao, Mingjie Tang, and Ji-Rong Wen. ClawGym II: Exploring black-box RL on agent harness, 2026a. URL https://arxiv.org/abs/2608.16798.

Huatong Song, Lisheng Huang, Shuang Sun, Jinhao Jiang, Ran Le, Daixuan Cheng, Guoxin Chen, Yiwen Hu, Zongchao Chen, Yiming Jia, Wayne Xin Zhao, Yang Song, Tao Zhang, and Ji-Rong Wen. SWE-Master: Unleashing the potential of software engineering agents via post-training, 2026b. URL https://arxiv.org/abs/2602.03411.

Zhiqiang Tan, Maoxin Wang, Sijie Wang, Yiming Yin, Qiang Wang, Xiaowen Chu, and Shaohuai Shi. Bidirectional resource scheduling for disaggregated and

asynchronous RL post-training, 2026. URL https://arxiv.org/abs/2607.09207.

Jinghui Wang, Shaojie Wang, Yinghan Cui, Xuxing Chen, Chao Wang, Liang Huang, Can Tang, Xiaojiang Zhang, Junyi Peng, Li Wan, Haotian Zhang, and Bin Chen. Tree training: Accelerating agentic LLMs training via shared prefix reuse, 2025a. URL https://arxiv.org/abs/2511.00413.

Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Zihe Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library, 2025b. URL https://arxiv.org/abs/2506.06122.

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Fei-Fei Li, Lijuan Wang, Yejin Choi, and Manling Li. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning, 2025c. URL https://arxiv.org/abs/2504.20073.

Tianyuan Wu, Lunxi Cao, Yining Wei, Wei Gao, Yuheng Zhao, Dakai An, Shaopan Xiong, Zhiqiang Lv, Ju Huang, Siran Yang, Yinghao Yu, Jiamang Wang, Lin Qu, and Wei Wang. Weave: Efficient Co-Scheduling for disaggregated RL Post-Training. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pp. 809–827, Seattle, WA, 2026a. USENIX Association. ISBN 978-1-939133-55-7. URL https://www.usenix.org/conference/osdi26/presentation/wu-tianyuan.

Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z. Morley Mao, Arvind Krishnamurthy, and Ion Stoica. RLBoost: Harvesting preemptible cloud resources for cost-efficient reinforcement learning on LLMs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pp. 1323–1339, Renton, WA, 2026b. USENIX Association. ISBN 978-1-939133-54-0. URL https://www.usenix.org/conference/nsdi26/presentation/wu-yongji.

Binfeng Xu, Hao Zhang, Shaokun Zhang, Songyang Han, Mingjie Liu, Jian Hu, Shizhe Diao, Zhenghui Jin, Yunheng Zou, Michael Demoret, Jan Kautz, and Yi Dong. Polar: Agentic RL on any harness at scale, 2026a. URL https://arxiv.org/abs/2605.24220.

Tianqi Xu, Lu Lv, Haoyang Huang, Wenjie Huang, Zhanming Shen, Yuhao Shen, Baolin Zhang, Xinyi Hu, Shuang Ge, Jun Dai, Tianyu Liu, Suorong Yang, Zhikai Li, Ye Bai, Jun Zhang, Lei Chen, Yue Li, and Mingchen Wan. TailSieve: Partial-rollout-guided tail routing for LLM rollouts, 2026b. URL https://arxiv.org/abs/2608.22788.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pp. 50528–50652. Curran Associates, Inc., 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. ProgramBench: Can language models rebuild programs from scratch?, 2026. URL https://arxiv.org/abs/2605.03546.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Mianjie Yu, Zizhao Mo, Huanyu Qu, Zhirong Qian, Huanle Xu, Cen Li, Zifeng Zhao, Zhi Zhou, Jinhua Zhou, Jun Xie, and Chengzhong Xu. psRL: Efficient training for agentic AI via training-time prefix sharing, 2026. URL https://arxiv.org/abs/2608.25683.

Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Hongning Wang, Jie Tang, and Yuxiao Dong. AgentRL: Scaling agentic reinforcement learning with a multi-turn, multi-task framework, 2025. URL https://arxiv.org/abs/2510.04206.

Lei Zhang, Mouxiang Chen, Ruisheng Cao, Jiawei Chen, Fan Zhou, Yiheng Xu, Jiaxi Yang, Zeyao Ma, Liang Chen, Changwei Luo, Kai Zhang, Fan Yan, KaShun Shum, Jiajun Zhang, Zeyu Cui, Feng Hu, Junyang Lin, Binyuan Hui, and Min Yang. MegaFlow: Large-scale distributed orchestration system for the agentic era, 2026a. URL https://arxiv.org/abs/2601.07526.

Zili Zhang, Yinmin Zhong, Chengxu Yang, Chao Jin, Bingyang Wu, Xinming Wei, Yuliang Liu, and Xin Jin. Heddle: A distributed orchestration system for agentic RL rollout, 2026b. URL https://arxiv.org/abs/2603.28101.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization, 2025. URL https://arxiv.org/abs/2507.18071.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark

Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, pp. 62557–62583. Curran Associates, Inc., 2024. doi: 10.52202/079017-2000. URL https://doi.org/10.52202/079017-2000.

Yinmin Zhong, Zili Zhang, Xiaoniu Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, Hongyu Zhou, Yimin Jiang, Yibo Zhu, and Daxin Jiang. StreamRL: Scalable, heterogeneous, and elastic RL for LLMs with disaggregated stream generation, 2025. URL https://arxiv.org/abs/2504.15930.

Yuzhen Zhou, Jiajun Li, Yusheng Su, Gowtham Ramesh, Zilin Zhu, Xiang Long, Chenyang Zhao, Jin Pan, Xiaodong Yu, Ze Wang, Kangrui Du, Jialian Wu, Ximeng Sun, Jiang Liu, Qiaolin Yu, Hao Chen, Zicheng Liu, and Emad Barsoum. APRIL: Active partial rollouts in reinforcement learning to tame long-tail generation, 2025. URL https://arxiv.org/abs/2509.18521.

Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents, 2024. URL https://arxiv.org/abs/2410.10934.

A Mean Staleness under Boundary Dispatch

All three methods in Section 6 use boundary dispatch. We derive the mean staleness used to match their configurations and explain why the identity also holds under elastic GPU placement.

A.1 Model and Accounting Assumptions

Each training step consumes B distinct rollout groups. Policy versions are indexed by training steps and increase by one after each optimizer update. A burst contains μ steps and publishes the weights from its final step with that step's version number. Following Section 2.2, define a consumed group's staleness as

$$d = v_t - v_d, \tag{8}$$

where v_d is the latest published version at admission and v_t is the trainer version immediately before the consuming update. Admission may precede execution launch (Section 4.2). This metric includes queueing and readiness waiting, but does not track the behavior policy of each model call. Means weight each consumed group equally.

Under boundary dispatch, every admitted group retains its slot until consumption. The queue is filled at startup and replenished with μB new groups after each burst's publication. Its target depth is

$$Q_b = (\varphi + \mu)B, \tag{9}$$

where $\varphi \geq 0$ is extra dispatch in units of B groups. Q_b counts all unconsumed groups after refill, including queued, running, and ready groups. No new groups are admitted within a burst.

Assumption A.1 (Stationary group accounting)**.** B, μ, and Q_b are fixed. Every admitted group is eventually consumed exactly once. At corresponding burst boundaries, the unconsumed population and its relative version ages are stationary, with finite expected total outstanding age.

A.2 Boundary Staleness Identity

Theorem A.1 (Mean staleness)**.** *Under Assumption A.1,*

$$\mathbb{E}_b[d] = \frac{Q_b}{B} - \frac{\mu+1}{2} = \varphi + \frac{\mu-1}{2}. \tag{10}$$

Proof. After update j in a burst, $Q_b - jB$ groups remain, and each gains one unit of age. Replacements enter only after publication, at age zero. Stationarity equates expected age removed by consumption with expected age added by updates, so

$$\mu B\, \mathbb{E}_b[d] = \sum_{j=1}^{\mu} (Q_b - jB) = \mu Q_b - \frac{B\mu(\mu+1)}{2}.$$

Dividing by μB and substituting $Q_b = (\varphi + \mu)B$ proves the result. □

The proof counts admission and consumption events, so neither FIFO consumption nor a particular rollout latency distribution is needed. Moving GPUs between rollout and training changes event timing, but preserves this identity as long as the boundary-dispatch accounting is unchanged. It therefore applies to Async, Colocate, and QWENGYRE.

A.3 Configurations at a Common Mean

For a target mean s, the identity gives

$$\varphi = s - \frac{\mu-1}{2} \geq 0,$$
$$Q_b = B\left(s + \frac{\mu+1}{2}\right).$$

The burst size must satisfy $\mu \leq 2s+1$, and group counts must be integral. Larger feasible bursts permit a greater dispatch depth at the same mean; their performance still depends on placement and workload latency.

With one step per burst, Async has $s = \varphi$. Without extra dispatch, Colocate has $s = (\mu-1)/2$. Thus, at $s = 1.5$, the (φ, μ) settings $(1.5, 1)$ for Async, $(0, 4)$ for Colocate, and $(0.5, 3)$ for QWENGYRE share the same mean staleness. The same calculation preserves the mean when the scheduling ablation changes QWENGYRE from $(0.5, 3)$ to $(1.5, 1)$.

B RL Algorithm Details

This appendix specifies the GSPO adaptation summarized in Section 6.1, including its execution-level clipping, token-level importance correction, loss normalization, and training hyperparameters.

Training configuration. We use Group Sequence Policy Optimization (GSPO) (Zheng et al., 2025) with token-level importance correction. Each rollout group contains $N = 16$ independent rollout executions. Each training step consumes a batch of $B = 32$ distinct rollout groups for NL2RepoBench or $B = 48$ for DeepSWE and TerminalBench. The remaining RL hyperparameters are shared across all experiments.

All configurations use Adam (Kingma & Ba, 2015) with a constant learning rate of 10^{-6}, $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-15}$, weight decay 0.01, and gradient clipping at norm 1.0. KL and entropy regularization are disabled.

Execution-level targets. For execution i, let $\mathcal{R}_i$ be its admitted trajectories. For each $r \in \mathcal{R}_i$, the target set $\mathcal{T}_r$ is fixed to the trainable tokens still unmasked when r is drawn (Section 5.3). On-tree masking makes these sets disjoint. Let $\mathcal{T}_i = \bigcup_{r\in\mathcal{R}_i} \mathcal{T}_r$, $L_r = |\mathcal{T}_r|$, and $L_i = |\mathcal{T}_i| = \sum_{r\in\mathcal{R}_i} L_r$. The sequence used for importance-ratio computation and clipping is this entire execution's set of admitted, deduplicated trainable tokens. Each token retains its original generation context. Tokens excluded by provenance, role, or the $J_{\max}$ cutoff contribute neither to the sequence ratio nor to the loss.

Importance correction and clipping. Let y_u be target token u, c_u its recorded conditioning context, and p_u^{roll} its recorded behavior probability, reused without recomputation. Let sg denote stop-gradient. For $L_i > 0$, the token importance ratio and detached execution-level sequence ratio are

$$\begin{aligned} \rho_u(\theta) &= \frac{\pi_\theta(y_u \mid c_u)}{p_u^{\text{roll}}}, \\ s_i(\theta) &= \operatorname{sg}\left[\exp\left(\frac{1}{L_i}\sum_{u\in\mathcal{T}_i}\log\rho_u(\theta)\right)\right]. \end{aligned} \tag{11}$$

The detached sequence ratio in Equation 11 is used for monitoring and determines a shared clipping mask according to execution i's group-relative advantage $\hat{A}_i$:

$$m_i(\theta) = \begin{cases} 0, & \hat{A}_i > 0 \text{ and } s_i(\theta) > 1.005, \\ 0, & \hat{A}_i < 0 \text{ and } s_i(\theta) < 0.995, \\ 1, & \text{otherwise.} \end{cases} \tag{12}$$

The token-specific importance weight is $\bar{\rho}_u(\theta) = \operatorname{sg}(\min\{\rho_u(\theta), 5\})$. These weights may differ within an execution; s_i affects the loss only through the common mask in Equation 12. The token loss, upper-bounded by 100, is

$$\ell_u(\theta) = \min\left\{100,\ -m_i(\theta)\bar{\rho}_u(\theta)\hat{A}_i \log \pi_\theta(y_u \mid c_u)\right\}. \tag{13}$$

Loss normalization. Using the token losses in Equation 13, weighting each trajectory by its target share L_r/L_i and averaging over its targets gives

$$\mathcal{L}_i(\theta) = \sum_{r\in\mathcal{R}_i}\frac{L_r}{L_i}\cdot\frac{1}{L_r}\sum_{u\in\mathcal{T}_r}\ell_u(\theta) = \frac{1}{L_i}\sum_{u\in\mathcal{T}_i}\ell_u(\theta), \tag{14}$$

Equation 14 gives every admitted target the same normalization coefficient $1/L_i$; importance weighting and clipping do not change this denominator. Executions with $L_i = 0$ contribute zero loss. A batch averages over all B queries and their N original rollout executions:

$$\mathcal{L}(\theta) = \frac{1}{BN}\sum_{q=1}^{B}\sum_{k=1}^{N}\mathcal{L}_{i(q,k)}(\theta), \tag{15}$$

where $i(q,k)$ is query q's k-th execution. Equation 15 preserves each execution's aggregate normalization weight regardless of how many trajectories it contributes. For a fixed admitted target set $\mathcal{T}_i$, changing which selected path carries a shared target leaves the execution's sequence ratio, clipping decision, and loss unchanged. Sampling can change $\mathcal{T}_i$ itself; the invariance concerns the assignment and ordering of a fixed set of targets. Packing preserves these per-execution weights.

C Case Study: One Execution, Multiple Training Trajectories

We examine execution `f3ad38e5`, in which an agent uses Claude Code to build a Python analytics SDK from a natural-language specification. The task requires nine packages, including catalog services, data frames, Flight connectivity, and pipeline utilities, implemented from scratch in a shared workspace. Figure 10 combines its prefix structure with recorded messages to illustrate the trajectory-processing problem of Section 5.

C.1 Execution and Context Structure

The trace reports 636 proxy requests and 1,347 unique message nodes across ten candidate trajectories. The main agent produces two task-driving trajectories and one summary. Five sub-agent invocations produce six task-driving trajectories and one summary: the catalog sub-agent compacts its history, while the FDW work item has two attempts. These visualization nodes count messages, distinct from the data plane's request increments and trainable tokens.

Main and sub-agent contexts have separate roots because their system prompts differ. Delegation can be identified separately by matching an `Agent` call's prompt to a sub-agent's initial user message after removing harness reminders; the export also identifies a sub-agent billing flag. Causal parentage therefore differs from context-prefix sharing. Concatenating the two agents' conversations would give outputs conditioning messages absent during generation.

C.2 Compaction and Shared Prefixes

Both the main agent and the catalog sub-agent compact their contexts. After their system prompts, they have shared prefixes of 168 and 193 messages, respectively. Each prefix forks into a task-driving assistant response and a summarization request–response pair. Continuation after compaction starts a separate path under the system prompt, with a new user message containing the summary.

The ten complete paths contain 53–293 message nodes each. Expanding them independently produces 1,716 node occurrences, compared with 1,347 shared nodes. The 369 extra occurrences repeat shared prefixes and system prompts; their token volume depends on message lengths.

If a task-driving leaf and its summary leaf are both selected, their shared policy outputs must contribute loss once. Section 5.3's on-tree masking retains the prefix as context while suppressing repeated targets. Provenance also matters: the original summary can be a target if the rollout policy generated it, whereas its reinsertion into the continuation's user prompt is conditioning input. TITO preserves the original token contexts and distinguishes these two occurrences.

C.3 Delegation, Interruption, and Outcome Validity

One response attempts four `Agent` calls, but the harness admits only two concurrent sub-agents. Two tool results report the concurrency limit. Subsequent calls start the remaining work, but the first FDW attempt is interrupted after a 53-node path. Later, a local test run reports `OK` for 259 tests, with one skipped, yet an import check cannot find the FDW, dbt, pipeline, and test-support packages. The main agent launches another attempt, which contributes a separate 202-node path. Rejection, partial execution, and retry thus affect the record differently within one rollout.

The leaf tagged `is_final_main` ends with renewed delegation: it identifies the last recorded main trajectory, not successful completion. An interrupted sub-agent likewise does not determine the execution's reward. Section 5.2 requires the designated evaluator to assess the preserved workspace; a group becomes trainable only after its executions close with valid outcomes. The visualization supplies no designated task reward. The recovery sequence also motivates preserving harness and workspace state during the request rerouting of Section 4.3.

C.4 Applying the Training Admission Rule

The priority classes are two *main*, one *main-summary*, six *sub-agent*, and one *sub-agent-summary*. For illustration, suppose $J_{\max} = 4$ and each path has eligible policy-generated targets. The rule in Section 5.3 admits both main trajectories, the main-summary trajectory, and one sub-agent trajectory. Selection within each class is proportional to remaining trainable tokens, with shared targets masked after every draw. The figure's message counts cannot determine these token probabilities.

A flat mean over all ten leaf losses would allocate eight equal coefficients to delegated or summary paths and repeat shared targets. In QWENGYRE, selected paths inherit the original execution's group-relative advantage. Token losses are averaged over disjoint targets within each execution, then over the original rollout executions. Additional compaction or delegation changes the admitted targets without creating independent rewards or increasing the execution's aggregate normalization weight.



Figure 10: Prefix graph of execution `f3ad38e5` with real message excerpts. Rectangles represent collapsed context segments; right-angle edges encode prefix sharing. Each parent is top-aligned with its first child, and other children are stacked below. Node counts refer to each segment; *path* counts the complete root-to-leaf context. Dashed rectangles identify summary leaves. Excerpts carry their original roles and node IDs; ellipses mark omissions. The interruption at `TOOL[1036]` remains in the main context where it was received.